

GUARÁ

Documentação Técnica do Aplicativo

Aplicativo de Segurança Pessoal com Inteligência Artificial

Disciplina: Trabalho Interdisciplinar — TI4

Curso: Ciência da Computação

Github: <https://github.com/ICEI-PUC-Minas-CC-TI/plmg-cc-ti4-2026-1-g11-guara>

Versão: 1.0.0

Data: Junho de 2026

Equipe do Projeto

O aplicativo Guará foi desenvolvido por uma equipe de seis estudantes do curso de Ciência da Computação, sob a orientação de quatro professores responsáveis.

Integrantes

- Daniel Matos Marques
- Davi Manoel Bernardes
- Felipe Costa Unsonst
- Felipe Quites Lopes
- Nayron Campos Soares
- Victor Monteiro Martinelli Grataroli

Professores Responsáveis

- Cristiano Neves Rodrigues
- Ilo Amy Saldanha Rivero
- Pedro Henrique Ramos Costa
- Rosilane Ribeiro da Mota

Sumário

1. Visão Geral do Projeto
2. Arquitetura e Tecnologias
3. Estrutura de Diretórios
4. Modelos de Dados
5. Banco de Dados Local (SQLite)
6. Sincronização em Nuvem (Firebase)
7. Controle de Sessão
8. Telas e Funcionalidades
9. Serviço de Análise de Risco — Gemini AI
10. Serviço de Alertas — Telegram
11. Permissões do Sistema
12. Paleta de Cores e Design
13. Dependências do Projeto
14. Fluxo de Funcionamento
15. Segurança e Privacidade
16. Próximos Passos e Melhorias Futuras

1. Visão Geral do Projeto

O Guará é um aplicativo de segurança pessoal desenvolvido com o objetivo de proteger usuários em situações de risco ou emergência. A proposta central é permitir que o usuário ative uma gravação de áudio do ambiente ao se sentir ameaçado. Esse áudio é transcrito em tempo real e analisado por uma Inteligência Artificial (Google Gemini) para detectar possíveis situações de perigo, como coação, intimidação, mal-estar médico ou invasão de espaço pessoal.

Quando um risco é detectado, o aplicativo envia alertas contendo a localização exata do usuário, a transcrição do áudio e a avaliação de risco para contatos de emergência pré-cadastrados, utilizando a API do Telegram. O sistema também armazena o histórico de gravações e os contatos de emergência de forma local (SQLite), com sincronização automática em nuvem (Firebase Firestore) quando há conexão com a internet.

1.1 Público-Alvo

O aplicativo é destinado a pessoas que desejam aumentar sua segurança no cotidiano, bem como a usuários que buscam garantir o bem-estar de amigos e familiares em situações de vulnerabilidade.

1.2 Objetivos de Desenvolvimento Sustentável (ODSs) Relacionadas

O projeto está alinhado com os seguintes Objetivos de Desenvolvimento Sustentável da ONU:

- ODS 3 — Saúde e Bem-Estar: Prevenção de acidentes, violência de gênero e promoção da segurança no trabalho e no cotidiano.
- ODS 11 — Cidades e Comunidades Sustentáveis: Foco na segurança física no ambiente urbano, tornando espaços públicos mais seguros.
- ODS 16 — Paz, Justiça e Instituições Eficazes: Promoção de sociedades pacíficas e redução de todas as formas de violência.

2. Arquitetura e Tecnologias

O aplicativo foi desenvolvido utilizando o framework Flutter, garantindo compatibilidade multiplataforma com Android, iOS, macOS, Windows e Linux a partir de uma única base de código em linguagem Dart. A arquitetura adota o padrão offline-first, onde o banco de dados local (SQLite) é a fonte de verdade, e a sincronização com a nuvem ocorre de forma assíncrona e silenciosa.

2.1 Stack Tecnológica

Camada	Tecnologia / Pacote	Finalidade
Framework	Flutter / Dart	Desenvolvimento multiplataforma (Android, iOS, macOS, Windows, Linux)
Armazenamento Local	SQLite (sqlite ^2.2.8)	Banco de dados relacional no dispositivo — fonte de verdade
Sincronização em Nuvem	Firebase Firestore (cloud_firestore ^5.4.4)	Backup e sincronização de dados entre dispositivos
Autenticação Firebase	firebase_auth ^5.3.1	Suporte a autenticação via Firebase (integração futura)
Sessão Local	SharedPreferences ^2.2.2	Persistência da sessão do usuário logado
Análise de Risco (IA)	Google Gemini API (http ^1.1.0)	Análise de texto para detecção de situações de perigo
Alertas de Emergência	Telegram Bot API (http ^1.1.0)	Envio de mensagens e localização para contatos
Geolocalização	geolocator ^14.0.2	Obtenção de coordenadas GPS do dispositivo
Reconhecimento de Voz	speech_to_text ^7.4.0	Transcrição de áudio em tempo real
Conectividade	connectivity_plus ^6.0.5	Verificação de conexão com a internet
Criptografia	crypto ^3.0.3	Hash de senhas dos usuários

2.2 Compatibilidade de Plataformas

Funcionalidade	Android	iOS	macOS	Windows	Linux
Geolocalização (GPS)	Sim	Sim	Sim	Sim	Sim
Reconhecimento de Voz (STT)	Sim	Sim	Parcial*	Parcial*	Parcial*
Gravação de Áudio	Sim	Sim	Sim	Sim	Sim
Acesso à Internet (APIs)	Sim	Sim	Sim	Sim	Sim
Análise com Gemini AI	Sim	Sim	Sim	Sim	Sim
Alertas via Telegram	Sim	Sim	Sim	Sim	Sim

* Em macOS, Windows e Linux, o STT tem limitações. A página de testes (testePage.dart) permite simular a entrada de texto.

3. Estrutura de Diretórios

O projeto segue a estrutura padrão do Flutter, com toda a lógica de negócios e interface de usuário contidas no diretório `lib/`. Abaixo está a organização dos arquivos mais relevantes:

```
Guara/
├── lib/
│   ├── main.dart                # Ponto de entrada, roteamento e inicialização
│   ├── colors.dart              # Paleta de cores do aplicativo
│   ├── models/
│   │   ├── user.dart           # Modelo de dados do usuário
│   │   ├── contact.dart        # Modelo de contato de emergência
│   │   ├── recording.dart       # Modelo de gravação e análise
│   │   └── analise_perigo.dart  # Modelo de resposta da IA
│   ├── data/
│   │   ├── database_helper.dart # CRUD SQLite e hash de senha
│   │   ├── cloud_sync.dart      # Sincronização com Firebase Firestore
│   │   └── session_helper.dart  # Controle de sessão ativa
│   ├── services/
│   │   ├── gemini_service.dart  # Integração com a API do Google Gemini
│   │   └── telegram_service.dart # Integração com a API do Telegram
│   └── pages/
│       ├── loginPage.dart       # Tela de login
│       ├── cadastropage.dart    # Tela de cadastro de usuário
│       ├── gravacaoPage.dart     # Tela principal de gravação e histórico
│       ├── contatosPage.dart     # Tela de listagem de contatos
│       ├── cadastroContatoPage.dart # Tela de cadastro/edição de contato
│       ├── perfilPage.dart       # Tela de perfil do usuário
│       └── notificacoesPage.dart # Tela de notificações (em
desenvolvimento)
│   └── InfoPage.dart            # Tela "Sobre o App"
├── assets/
│   └── images/                  # Logotipo e imagens do aplicativo
└── pubspec.yaml                 # Dependências e configurações do projeto
```

4. Modelos de Dados

O sistema é composto por quatro entidades principais que representam os dados manipulados pelo aplicativo. Cada entidade possui métodos de serialização (toMap / fromMap) para persistência no banco de dados e sincronização com o Firebase.

4.1 User (Usuário)

Representa o usuário cadastrado no aplicativo. A senha é armazenada como hash (SHA-256) e nunca em texto claro.

Campo	Tipo	Descrição
id	int?	Identificador único gerado pelo banco de dados (auto-increment)
name	String	Nome completo do usuário
email	String	E-mail único do usuário (normalizado para minúsculas)
password	String	Hash SHA-256 da senha do usuário
dateCreated	DateTime	Data e hora de criação do cadastro

4.2 Contact (Contato de Emergência)

Representa um contato de emergência vinculado a um usuário específico.

Campo	Tipo	Descrição
id	int?	Identificador único do contato
userId	int	Chave estrangeira referenciando o usuário proprietário
name	String	Nome do contato de emergência
relationship	String	Relação com o usuário (ex: Mãe, Amigo, Cônjuge)
phoneNumber	String	Número de telefone do contato
dateCreated	DateTime	Data de cadastro do contato

4.3 Recording (Gravação)

Armazena os dados de uma sessão de gravação, incluindo a transcrição do áudio e o resultado da análise de risco.

Campo	Tipo	Descrição
id	int?	Identificador único da gravação
userId	int	Chave estrangeira do usuário que realizou a gravação
date	DateTime	Data e hora da gravação
path	String	Caminho do arquivo de áudio (pode estar vazio)
transcription	String	Texto transcrito do áudio capturado
nivel	String?	Nível de perigo classificado pela IA: BAIXO, MEDIO ou ALTO

perigo	bool	Indica se a IA classificou a gravação como situação de perigo
---------------	------	---

4.4 AnalisePerigo (Análise de Risco)

Estrutura de dados que encapsula a resposta da API do Google Gemini após a análise de uma transcrição de áudio.

Campo	Tipo	Descrição
perigo	bool	Indica se foi detectada situação de perigo no texto analisado
nivel	String	Nível de risco: BAIXO, MEDIO ou ALTO
frase	String	Trecho principal do texto que indica o perigo detectado
confianca	double	Score de confiança da análise, de 0.0 (sem certeza) a 1.0 (certeza total)

5. Banco de Dados Local (SQLite)

O DatabaseHelper é implementado como um Singleton e gerencia todas as operações de leitura e escrita no banco de dados SQLite local. Ele utiliza o padrão lazy initialization, criando o banco apenas na primeira vez que é acessado. O banco de dados é composto por três tabelas: users, contacts e recordings.

5.1 Schema do Banco de Dados

```
-- Tabela de usuários
CREATE TABLE users (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  name        TEXT      NOT NULL,
  email       TEXT      NOT NULL UNIQUE,
  password    TEXT      NOT NULL,
  dateCreated TEXT      NOT NULL
);

-- Tabela de contatos de emergência
CREATE TABLE contacts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  userId      INTEGER NOT NULL,
  name        TEXT      NOT NULL,
  relationship TEXT      NOT NULL,
  phoneNumber TEXT      NOT NULL,
  dateCreated TEXT      NOT NULL
);

-- Tabela de gravações
CREATE TABLE recordings (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  userId      INTEGER NOT NULL,
  date        TEXT      NOT NULL,
  path        TEXT      NOT NULL DEFAULT '',
  transcription TEXT      NOT NULL DEFAULT '',
  nivel       TEXT,
  perigo      INTEGER NOT NULL DEFAULT 0
);
```

5.2 Operações Disponíveis

Entidade	Operação	Método	Descrição
User	Autenticar	authenticate(email, password)	Valida credenciais e retorna o usuário ou null
User	Inserir	insertUser(user)	Cria novo usuário com senha em hash e sincroniza na nuvem
User	Buscar por e-mail	getUserByEmail(email)	Localiza usuário pelo e-mail normalizado
Contact	Inserir	insertContact(contact)	Cria novo contato e sincroniza na nuvem
Contact	Atualizar	updateContact(contact)	Atualiza dados do contato e sincroniza
Contact	Excluir	deleteContact(id)	Remove contato local e da nuvem

Contact	Listar	getAllContacts({userId})	Retorna todos os contatos, opcionalmente filtrados por usuário
Recording	Inserir	insertRecording(recording)	Salva nova gravação e sincroniza na nuvem
Recording	Atualizar	updateRecording(recording)	Atualiza dados da gravação e sincroniza
Recording	Excluir	deleteRecording(id)	Remove gravação local e da nuvem
Recording	Listar	getAllRecordings({userId})	Retorna todas as gravações, opcionalmente por usuário

6. Sincronização em Nuvem (Firebase Firestore)

O CloudSync é implementado como um Singleton que atua como camada de sincronização com o Firebase Firestore. Sua filosofia de design é o princípio de falha silenciosa: se a nuvem estiver indisponível, o aplicativo continua funcionando normalmente com o banco de dados local, sem nenhuma interrupção para o usuário.

A inicialização do Firebase é tentada na inicialização do aplicativo (main.dart). Caso o Firebase não esteja configurado (ex.: em ambiente de desenvolvimento sem o arquivo google-services.json), o sistema retorna false e opera exclusivamente de forma local.

6.1 Estrutura de Coleções no Firestore

```
Firestore
├── users/
│   ├── {userId}/
│   │   ├── (campos: id, name, email, password, dateCreated)
│   │   ├── contacts/
│   │   │   ├── {contactId}/
│   │   │   │   ├── (campos: id, userId, name, relationship, phoneNumber,
│   │   │   │   │   dateCreated)
│   │   │   │   └── recordings/
│   │   │   │       ├── {recordingId}/
│   │   │   │       │   ├── (campos: id, userId, date, path, transcription, nivel,
│   │   │   │       │   │   perigo)
```

6.2 Lógica de Sincronização

Antes de qualquer operação de escrita na nuvem, o CloudSync verifica duas condições: (1) se o Firebase foi inicializado corretamente e (2) se há conectividade com a internet via `connectivity_plus`. Apenas se ambas as condições forem verdadeiras a operação de sincronização é executada. Todas as operações de escrita no Firestore utilizam `SetOptions(merge: true)`, garantindo que apenas os campos fornecidos sejam atualizados, sem sobrescrever dados existentes.

7. Controle de Sessão

O SessionHelper gerencia a sessão do usuário ativo utilizando o SharedPreferences para persistir as informações entre as sessões do aplicativo. Isso permite que o usuário permaneça logado mesmo após fechar o aplicativo.

Método	Descrição
<code>saveSession(userId, name, email)</code>	Persiste os dados da sessão ativa no SharedPreferences
<code>isLoggedIn()</code>	Verifica se há uma sessão ativa válida
<code>getLoggedInUserId()</code>	Retorna o ID do usuário logado
<code>getLoggedInUserName()</code>	Retorna o nome do usuário logado
<code>getLoggedInUserEmail()</code>	Retorna o e-mail do usuário logado
<code>clearSession()</code>	Encerra a sessão, removendo os dados de autenticação

O widget `_StartupGate`, no `main.dart`, é responsável por verificar a sessão ao iniciar o aplicativo. Se não houver sessão ativa, uma sessão padrão é criada automaticamente, redirecionando o usuário diretamente para a tela principal sem necessidade de login manual.

8. Telas e Funcionalidades

O aplicativo é organizado em uma navegação por abas na parte inferior da tela (BottomNavigationBar), com três seções principais: Contatos, Gravar e Perfil. A tela de Notificações está implementada mas comentada na versão atual, aguardando desenvolvimento futuro.

8.1 Tela de Login (loginPage.dart)

Tela de autenticação do usuário. Permite o login com e-mail e senha, com validação de formulário. Inclui um link para a tela de cadastro de novos usuários. A autenticação é realizada consultando o banco de dados local SQLite.

8.2 Tela de Cadastro (cadastropage.dart)

Permite o registro de novos usuários. O formulário valida nome, e-mail (formato), senha (mínimo 6 caracteres) e confirmação de senha. Ao cadastrar, a senha é convertida em hash antes de ser armazenada.

8.3 Tela de Gravação (gravacaoPage.dart)

Esta é a tela central do aplicativo, dividida em duas partes: a GravacaoPage (histórico de gravações) e a GravacaoSessionPage (sessão de gravação ativa). A GravacaoPage exibe a lista de gravações anteriores com indicadores visuais de nível de perigo. A GravacaoSessionPage é responsável por:

- Obter a localização GPS do usuário com alta acurácia (LocationAccuracy.best).
- Inicializar e gerenciar o reconhecimento de voz em tempo real.
- Acumular a transcrição de forma permanente, mesmo com reinicializações do motor de voz.
- Enviar a transcrição para análise de risco pela IA a cada 10 segundos.
- Exibir indicadores visuais de perigo detectado em tempo real.
- Ao finalizar, salvar a gravação no banco de dados e enviar alertas via Telegram.

8.4 Tela de Contatos (contatosPage.dart)

Exibe a lista de contatos de emergência cadastrados pelo usuário. Cada contato é apresentado em um card com nome, relação e telefone. A tela oferece opções para adicionar novos contatos, editar e excluir contatos existentes, com confirmação antes da exclusão.

8.5 Tela de Cadastro de Contato (cadastroContatoPage.dart)

Formulário para adicionar ou editar um contato de emergência. Os campos obrigatórios são: nome, relação e número de telefone. A tela funciona tanto para criação quanto para edição, recebendo um objeto Contact opcional como parâmetro.

8.6 Tela de Perfil (perfilPage.dart)

Exibe as informações do usuário logado (nome e e-mail). Oferece acesso à tela 'Sobre o App' (InfoPage) e a opção de logout, que encerra a sessão e redireciona para a tela de login.

8.7 Tela Sobre o App (InfoPage.dart)

Página informativa que apresenta o nome do aplicativo, logotipo e a versão atual (1.0.0). Serve como referência para o usuário sobre o aplicativo instalado.

9. Serviço de Análise de Risco — Gemini AI

O GeminiService é responsável por enviar a transcrição de áudio para a API do Google Gemini e interpretar a resposta de análise de risco. O serviço utiliza o modelo gemini-1.5-flash-lite, configurado para responder exclusivamente em formato JSON, com temperatura 0.0 para garantir respostas determinísticas.

9.1 Prompt de Sistema

O Gemini recebe uma instrução de sistema (system instruction) que define seu papel como analisador de transcrições de áudio de um aplicativo de segurança pessoal. O modelo é instruído a identificar os seguintes tipos de situação de risco:

- Coação e desvio de rota forçado.
- Intimidação velada ou abuso verbal.
- Mal-estar médico urgente.
- Invasão de espaço pessoal ou assalto.
- Qualquer outro sinal de perigo real.

9.2 Formato de Resposta da IA

A API retorna um JSON estruturado com os seguintes campos:

```
{
  "perigo": true,
  "nivel": "ALTO",
  "frase": "para de puxar meu braço",
  "confianca": 0.95
}
```

9.3 Parâmetros de Configuração da API

Parâmetro	Valor	Descrição
model	gemini-1.5-flash-lite	Modelo utilizado para análise de texto
responseMimeType	application/json	Força a resposta em formato JSON puro
temperature	0.0	Respostas determinísticas, sem variação aleatória
topP	1.0	Considera todos os tokens possíveis na geração
maxOutputTokens	120	Limita o tamanho da resposta para eficiência
timeout	10 segundos	Tempo máximo de espera pela resposta da API

9.4 Análise em Tempo Real

Durante uma sessão de gravação ativa, a análise de risco é executada automaticamente a cada 10 segundos (intervalo configurável via _maxIntervalAnalise). Se a transcrição não mudou desde a última análise, a chamada à API é ignorada para evitar requisições desnecessárias. Quando um perigo é

detectado, a interface exibe um indicador visual de alerta e o botão de salvar muda para 'Enviar (Perigo Detectado!)'.

10. Serviço de Alertas — Telegram

O TelegramService encapsula a comunicação com a API do Telegram Bot, permitindo o envio de mensagens de texto formatadas e localização geográfica para um chat específico. O serviço é instanciado com um botToken (obtido via @BotFather) e um chatId (ID do chat de destino).

10.1 Métodos Disponíveis

Método	Parâmetros	Descrição
enviarMensagem(mensagem)	mensagem: String	Envia mensagem de texto formatada em HTML para o chat configurado
enviarLocalizacao(...)	latitude: double, longitude: double	Envia um pin de localização no mapa do Telegram
enviarDadosCompleto(...)	mensagem, latitude, longitude	Envia mensagem de texto seguida da localização (com delay de 500ms entre os envios)

10.2 Formato da Mensagem de Alerta

Quando um alerta é disparado, a mensagem enviada ao Telegram segue o formato abaixo:

```
🚨 ALERTA DE PERIGO

🗣️ Transcrição:
"para de puxar meu braço"

🔴 Nível: ALTO
🎯 Frase chave: "para de puxar meu braço"
📊 Confiança: 95.0%
🕒 Horário: 14:23

[Mensagem de localização com pin no mapa]
```

10.3 Configuração

Para ativar o envio de alertas, é necessário configurar o botToken e o chatId na função _enviarParaTelegram() no arquivo gravacaoPage.dart, e descomentar a chamada ao método. Recomenda-se, para produção, mover essas credenciais para variáveis de ambiente.

11. Permissões do Sistema

Para o funcionamento completo do aplicativo, as seguintes permissões são necessárias e devem ser concedidas pelo usuário em tempo de execução:

11.1 Android (AndroidManifest.xml)

Permissão	Finalidade
ACCESS_FINE_LOCATION	GPS de alta precisão para obter coordenadas exatas do usuário
ACCESS_COARSE_LOCATION	GPS aproximado (fallback quando GPS preciso não está disponível)
RECORD_AUDIO	Gravação de áudio e reconhecimento de fala (STT)
INTERNET	Comunicação com as APIs do Gemini, Telegram e Firebase

11.2 iOS e macOS (Info.plist)

Chave	Descrição para o Usuário
NSLocationWhenInUseUsageDescription	O app precisa de acesso à sua localização para enviar informações de emergência.
NSLocationAlwaysAndWhenInUseUsageDescription	O app precisa de acesso à sua localização para enviar informações de emergência.
NSMicrophoneUsageDescription	O app precisa de acesso ao microfone para gravar áudios.
NSSpeechRecognitionUsageDescription	O app precisa de acesso ao reconhecimento de fala para transcrever áudios.

12. Paleta de Cores e Design

O aplicativo utiliza uma identidade visual baseada em tons de verde, transmitindo segurança e confiança. As cores são definidas como constantes globais no arquivo colors.dart:

Constante	Valor Hexadecimal	Uso Principal
corEscura	#004725	Títulos, bordas e elementos de destaque primário
corPrincipal	#007A3F	Botões, ícones ativos, cabeçalhos de tabela e cor de seleção
corDestaque	#02AD5A	Elementos de fundo suave, indicadores de atividade
corBackground	#ECF0F3	Cor de fundo geral das telas do aplicativo

13. Dependências do Projeto

As dependências do projeto são gerenciadas pelo arquivo pubspec.yaml. A versão atual do aplicativo é 1.0.0+1, desenvolvida para o SDK Dart ^3.11.1.

Pacote	Versão	Finalidade
sqflite	^2.2.8+1	Banco de dados SQLite para armazenamento local
path_provider	^2.0.15	Localização de diretórios no sistema de arquivos
path	^1.8.4	Manipulação de caminhos de arquivos
shared_preferences	^2.2.2	Armazenamento de preferências e sessão do usuário
crypto	^3.0.3	Hash SHA-256 para senhas
firebase_core	^3.6.0	Inicialização do Firebase
cloud_firestore	^5.4.4	Banco de dados em nuvem (Firebase Firestore)
firebase_auth	^5.3.1	Autenticação via Firebase (suporte futuro)
connectivity_plus	^6.0.5	Verificação de conectividade com a internet
speech_to_text	^7.4.0	Reconhecimento de voz em tempo real
geolocator	^14.0.2	Obtenção de coordenadas GPS
http	^1.1.0	Requisições HTTP para as APIs do Gemini e Telegram
cupertino_icons	^1.0.8	Ícones no estilo iOS
flutter_launcher_icons	^0.14.4	Geração de ícones do aplicativo para todas as plataformas

14. Fluxo de Funcionamento

14.1 Fluxo de Inicialização do Aplicativo

Ao ser iniciado, o aplicativo executa as seguintes etapas em sequência:

1. Inicialização do Flutter (`WidgetsFlutterBinding.ensureInitialized`).
2. Abertura do banco de dados SQLite local (`DatabaseHelper.instance.database`).
3. Tentativa de inicialização do Firebase (`CloudSync.instance.init`) — silenciosa em caso de falha.
4. Verificação da sessão ativa pelo `_StartupGate`.
5. Redirecionamento para a `HomePage` (se logado) ou criação de sessão padrão.

14.2 Fluxo de uma Sessão de Gravação e Alerta

O fluxo completo de uma sessão de gravação e envio de alerta é descrito abaixo:

- 1. Abertura da tela:** O usuário navega para a aba 'Gravar' e inicia uma sessão.
- 2. Obtenção de localização:** O GPS é ativado com alta acurácia (`LocationAccuracy.best`) e as coordenadas são armazenadas.
- 3. Inicialização do STT:** O motor de reconhecimento de voz é inicializado e começa a escutar.
- 4. Transcrição em tempo real:** O áudio é transcrito continuamente. O texto permanente é acumulado mesmo com reinicializações do motor.
- 5. Análise periódica:** A cada 10 segundos, a transcrição é enviada ao Gemini para análise de risco.
- 6. Detecção de perigo:** Se o Gemini detectar risco, a interface exibe alerta visual e o botão muda para 'Enviar (Perigo Detectado!)'.
- 7. Finalização:** O usuário toca no botão para parar a gravação e salvar.
- 8. Análise final:** Uma análise final é realizada com toda a transcrição acumulada.
- 9. Envio do alerta:** Os dados (localização + transcrição + análise) são formatados e enviados ao Telegram.
- 10. Persistência:** A gravação é salva no SQLite e sincronizada com o Firebase Firestore.

15. Segurança e Privacidade

O aplicativo implementa diversas práticas de segurança para proteger os dados do usuário:

Medida de Segurança	Status	Descrição
Hash de senhas (SHA-256)	Implementado	Senhas nunca são armazenadas em texto claro no banco de dados local ou na nuvem.
Permissões sob demanda	Implementado	Permissões de hardware (GPS, microfone) são solicitadas apenas no momento de uso.
Sandbox macOS	Implementado	O aplicativo opera dentro do sandbox de segurança do macOS.
HTTPS obrigatório	Implementado	Toda comunicação com APIs externas utiliza HTTPS.
Chaves de API no código	Pendente	As chaves do Gemini e Telegram estão hardcoded. Devem ser movidas para variáveis de ambiente em produção.
Criptografia de dados locais	Pendente	Os dados armazenados no SQLite não estão criptografados. Recomenda-se implementar criptografia em produção.
Autenticação de acesso à nuvem	Pendente	Implementar autenticação Firebase para proteger o acesso aos dados no Firestore.

16. Próximos Passos e Melhorias Futuras

Com base na análise do código e na documentação interna do projeto, as seguintes melhorias são recomendadas para as próximas iterações de desenvolvimento:

- 1. Gestão de Credenciais:** Mover as chaves de API do Gemini e do Telegram para arquivos de variáveis de ambiente (.env), eliminando o risco de exposição acidental em repositórios de código.
- 2. Fila de Envio com Retentativas:** Implementar um mecanismo de fila para o envio de alertas ao Telegram, com retentativas automáticas em caso de falha de rede, garantindo que nenhum alerta seja perdido.
- 3. Criptografia de Dados Locais:** Adicionar criptografia ao banco de dados SQLite para proteger os dados sensíveis armazenados no dispositivo em caso de acesso não autorizado.
- 4. Notificações de Status:** Implementar notificações push no dispositivo informando o usuário sobre o status de entrega dos alertas enviados.
- 5. Configuração Dinâmica do Telegram:** Criar uma interface de configuração para que o usuário possa inserir seu próprio botToken e chatId do Telegram diretamente pelo aplicativo.
- 6. Ativação da Tela de Notificações:** Desenvolver e ativar a NotificacoesPage, que atualmente está comentada na navegação principal.
- 7. Autenticação Firebase Completa:** Integrar o firebase_auth para substituir o sistema de autenticação local por um sistema robusto baseado em nuvem.
- 8. Contato Direto com Emergências:** Implementar a funcionalidade de contato automático com serviços de emergência (polícia, SAMU) com abertura de chamado, conforme previsto na concepção original do projeto.

17. Demonstração Funcional do App

A demonstração funcional tem como objetivo comprovar a viabilidade da solução proposta, evidenciando que os requisitos definidos para o projeto foram implementados e que o sistema está apto a atender às necessidades identificadas durante a fase de levantamento de requisitos.

Vídeo de demonstração: <https://www.youtube.com/shorts/eRI3tZZWBGY>
(O vídeo também está no github)

— *Fim da Documentação* —

Guará v1.0.0 | Junho de 2026 | plmg-cc-ti4-2026-1-g11